Document number	P2603R1
Date	2022-09-01
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Evolution Working Group (EWG)

member function pointer to function pointer

Table of contents

- member function pointer to function pointer
  - Abstract
  - Motivating example
    - Scenario #1 - Unknown State
    - Scenario #2 - Known State
      - Scenario #2a - Known State - Exact
      - Scenario #2b - Known State - Derived
  - Solution
  - Summary
  - Prior work
  - References

Changelog

R1

- shortened `member_function_pointer_to_free_function_pointer` to `to_free_function_pointer`
- clarified `consteval`
- added support of free function pointer passthrough

Abstract

`C++` allows one to call a base member function from a derived instance at compile time.

`Working Draft, Standard for Programming Language C++` [1]

"*11.7.3 Virtual functions [class.virtual]*"

"*16 Explicit qualification with the scope operator (7.5.4.3) suppresses the virtual call mechanism."*

"*[Example 10:"*

```
class B { public: virtual void f(); };
class D : public B { public: void f(); };

void D::f() { /* ... */ B::f(); }
```

“Here, the function call in D::f really does call B::f and not D::f. — end example]”

Currently `C++` **does not** allow calling a base member function from a derived instance at runtime. The member function pointer always resolves to the member function of the derived instance. While this does make sense for traditional runtime polymorphism, this behavior is less desired when selecting member functions for other types of runtime polymorphism as it occurs a superfluous dual dispatch.

Just as the programmer decides whether to call the virtual function virtually or not at compile time, the programmer should be able to do the same at runtime.

Motivating Example

```
class Abstract
{
public:
    virtual void some_virtual_function() = 0;
    virtual void some_virtual_function() const = 0;
};

class Base : public Abstract
{
public:
    void some_virtual_function() override { }
    void some_virtual_function() const override { }
};

class Derived : public Base
{
public:
    void some_virtual_function() override { }
    void some_virtual_function() const override { }
    void some_deducing_this_member_function(this Derived) { }
};

int main()
{
    Derived derived;
    Derived* pderived = &derived;
    Derived& rderived = derived;

    derived.some_virtual_function(); // Direct call to Derived::some_virtual_funct
    pderived->some_virtual_function(); // Virtual call in general case to Derived:
    rderived.some_virtual_function(); // Virtual call in general case to Derived::

    // NOTE: A derived class can call its base class member function directly
    // at compile time using a qualifier in these cases Base::
    derived.Base::some_virtual_function(); // Direct call to Base::some_virtual_fu
    pderived->Base::some_virtual_function(); // Direct call to Base::some_virtual_
    rderived.Base::some_virtual_function(); // Direct call to Base::some_virtual_f

    void (Base::*bmfp)() = &Base::some_virtual_function;
    void (Derived::*dmfp)() = &Derived::some_virtual_function;
    void (Derived::*dmfpc)() const = static_cast<void (Derived::*)() const>(&Deriv

    // NOTE: A derived class can NOT call its base class member function at runtim
    (derived.*bmfp)();// Derived::some_virtual_function
    (derived.*dmfp)();// Derived::some_virtual_function
    (derived.*dmfpc)();// Derived::some_virtual_function const
```

```
    return 0;
}

void Base_some_virtual_function(Base& instance)
{
     // Direct call to Base::some_virtual_function
    instance.Base::some_virtual_function();
}

void const_Base_some_virtual_function(const Base& instance)
{
     // Direct call to Base::some_virtual_function
    instance.Base::some_virtual_function();
}
```

The most relevant lines in question are the following:

```
Derived derived;

void (Base::*bmfp)() = &Base::some_virtual_function;

(derived.*bmfp)();// Derived::some_virtual_function
```

Even though the programmer explicitly stated that they want to call `Base`'s `some_virtual_function`, `Derived`'s `some_virtual_function` is always called. The programmer was never given the same choice that they have at compile time. Again this is correct for traditional runtime polymorphism. However, for a callback using raw pointers, `function_ref` [2], or `nontype function_ref` [3], the end programmer wants the choice and really should choose based on the scenario.

Scenario #1 - Unknown State

For instance, if the programmer received some pointer or reference to some instance and as such doesn't know the exact type of the instance than the logical and safe choice is to call the method virtually even though it incurs dual dispatch costs. This scenario **occurs frequently** when integrating with 3rd party libraries that are unaware of the callback type and as such the callback type is instantiated **late**.

```
function_ref<void()> factory(Base& base)
{
  // base could be of type Base, Derived or some other class not known at this fun
  // in which case dual dispatch is still a good idea
  function_ref<void()> fr = {nontype<&Base::some_virtual_function, some_virtual_ta
  return fr;
}
```

A library could provide an overload via a tag class, in this example `some_virtual_tag_class`, in order to allow the programmer to choose whether the function will be called virtually or directly, in this case the former.

Scenario #2 - Known State

However, if the programmer knows the instantiated type, likely because the programmer was the creator, then the programmer wants to avoid the superfluous cost of calling the member function through the member function pointer. This scenario **occurs even more frequently** when the programmer is calling his own code, his team member's code or a 3rd party library that is aware of the callback type and provide callback instances. As such the callback type is instantiated **early**.

Scenario #2a - Known State - Exact

In this scenario, the state is known and is the same type of the class that houses the member function. As such there is no need resolve the member function at runtime since the type is known.

```
Base base;
function_ref<void()> fr = {nontype<&Base::some_virtual_function, some_direct_tag_c
```

Again a library could provide an overload via a tag class, in this example `some_direct_tag_class`, in order to allow the programmer to choose whether the function will be called virtually or directly, in this case the later.

Scenario #2b - Known State - Derived

This would also work safely for derived state calling their base class member functions at runtime without the additional member function pointer costs. After all, `Derived` is still a `Base`.

```
Derived derived;
function_ref<void()> fr = {nontype<&Base::some_virtual_function, some_direct_tag_c
```

It should ne noted, that besides callbacks of single functions this functionality could be of benefit to callbacks of n-ary functions such as found in `proxy` [4], `dyno` [5] and `boost ext te` [6].

Solution

Unlike calling base class member functions with derived instances at compile time via a qualifier, it is **undesirable** to add keywords/vernacular/qualifiers at the point of function call, that is to choose direct or virtual at call time. This incurs additional runtime costs of potentially increased member function pointer size and execution time, even for those that don't need the choice as in traditional runtime polymorphism.

```
Derived derived;

void (Base::*bmfp)() = &Base::some_virtual_function;

// NOTE: This is NOT desired
(derived.*bmfp)() direct;// Base::some_virtual_function
(derived.*bmfp)() virtual;// Derived::some_virtual_function
```

It is also **undesirable** to add keywords/vernacular at the point of member function ponter initialization. This also incurs additional runtime costs of potentially increased member function pointer size and execution time, even for those that don't need the choice as in traditional runtime polymorphism.

```
Derived derived;

void (Base::*bmfp1)() = &Base::some_virtual_function direct;
void (Base::*bmfp2)() = &Base::some_virtual_function virtual;
```

What is desired is no change to member function pointer, at all! Rather, a new intrinsic `consteval` function would be created called `to_free_function_pointer` . This function would not take member function pointer at runtime but only a member function pointer initialization statement, `&class_name::member_function_name` , at compile time. Technically, it could also take a static_cast of a member function pointer initialization to a specific member function pointer type to allow explicit choosing of overloaded methods. What gets returned is just a free

function pointer that points to the actual member function or a thunk where the `this` reference is the first parameter. For `Deducing this` [7] member functions, the `this` type could be a value instead of a reference and as such this new function would just be a passthrough. This intrinsic function would also be overloaded for free functions in which case it would just be a passthrough.

```
Derived derived;
// initialized with member functions
void (*bfp1)(Base&) = to_free_function_pointer(&Base::some_virtual_function);
void (*dfp)(Derived&) = to_free_function_pointer(&Derived::some_virtual_function);
void (*dfpc)(const Derived&) = to_free_function_pointer(static_cast<void (Derived:
void (*ddtfp)(Derived) = to_free_function_pointer(&Derived::some_deducing_this_mem
// initialized with free functions (always passthrough)
void (*bfp2)(Base&) = to_free_function_pointer(Base_some_virtual_function);
void (*bfp3)(const Base&) = to_free_function_pointer(const_Base_some_virtual_funct
```

With this, `Base::some_virtual_function` can be called at runtime, simply initialized, like [member] function pointers, without having to use a lambda. The end result is member function pointers' initialization syntax can be used to select member functions with the knowledge that the selected is the one that actually will be called.

NOTE: The following is invalid code because there is no function when the member function declaration is pure.

```
void (*bfp)(Abstract&) = to_free_function_pointer(&Abstract::some_virtual_function
```

The brevity of the name of the intrinsic function is less relevant as it will in all likelihood be concealed in library implementations rather than used directly.

Summary

The advantages to `C++` with this proposal is manifold.

- A seemingly oversight in `C++` gets fixed by allowing calling a base member function from a derived instance at runtime
- Mitigates a bifurcation by allowing one to interact with member functions regardless of whether they are a `Deducing this` [7:1] member function or a legacy member function
- Mitigates another bifurcation by allowing one to interact with functions regardless of whether they are a member function or a free function

- Matches existing practice by allowing users to use member function pointer initialization to select member functions with the confidence that it is the function that will be called

Prior work

GCC has support acquiring the function pointer from a member function pointer via its `bound member function` [8] feature. While their implementation supports the functionality at both runtime and at compile time, this proposal is solely concerned about compile time. Also, while GCC uses a casting mechanism, this proposal is asking for a intrinsic function to better support automatic type deduction.

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/n4910.pdf) ↩

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r9.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0792r9.html) ↩

3. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p2472r3.html) ↩

4. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0957r7.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0957r7.pdf) ↩

5. https://github.com/ldionne/dyno (https://github.com/ldionne/dyno) ↩

6. https://github.com/boost-ext/te (https://github.com/boost-ext/te) ↩

7. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0847r7.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0847r7.html) ↩ ↩

8. https://gcc.gnu.org/onlinedocs/gcc/Bound-member-functions.html (https://gcc.gnu.org/onlinedocs/gcc/Bound-member-functions.html) ↩